



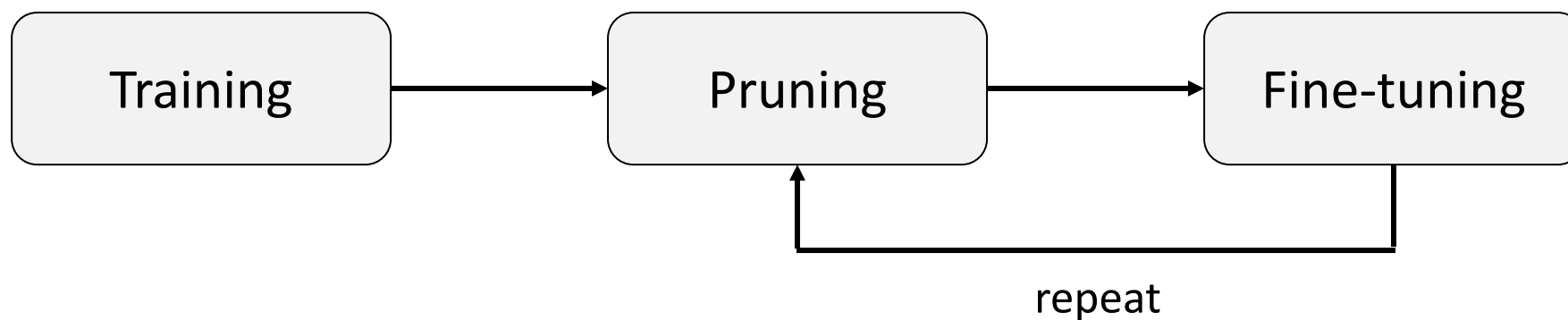
Lab4

Model Compression : Pruning & Quantization



Concept (pruning)

Network pruning is widely used for reducing the heavy inference cost of deep models in low-resource settings. A typical pruning algorithm is a three-stage pipeline.



Why pruning?

According to “Lottery Ticket Hypothesis” (Frankle & Carbin, 2019), we find that large network is easier to train than small network.



easy to train



hard to train



Method (pruning)

Pruning the channel of each convolution layer based on the weight of batch normalization.

$$\text{BN : } y_i^{(k)} = \gamma^{(k)} \hat{x}_i^{(k)} + \beta^{(k)}$$

The smaller the value of γ , the less important of $\hat{X}$.

```
for m in model.modules():
    if isinstance(m, nn.BatchNorm2d):
        size = m.weight.data.shape[0]
        bn[index:(index+size)] = m.weight.data.abs().clone()
        index += size

y, i = torch.sort(bn)
thre_index = int(total * pruning_rate)
thre = y[thre_index]
```



DATASET

Speech Commands

- [TensorFlow](#) released the Speech Commands Datasets. It includes 65,000 one-second long utterances of 30 short words, by thousands of different people.
- We only use 10 short words.

 down go left no off on right stop up yes

Network

SincNet

- SincNet is a neural architecture for processing **raw audio samples**.
- It is a novel Convolutional Neural Network (CNN) that encourages the first convolutional layer to discover more **meaningful filters**.

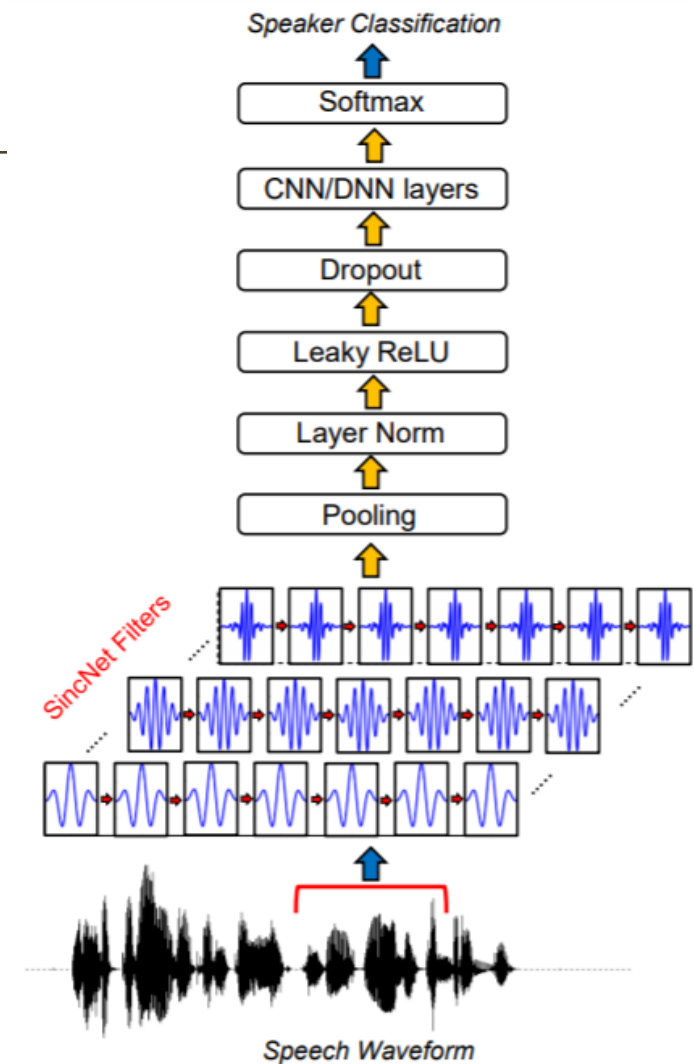


Fig. 1: Architecture of SincNet.

Task 1 (sincnet_training.ipynb)

Train SincNet

- Determine super parameter.

```
EPOCH = |  
BATCH_SIZE =  
LR =  
Weight_decay =
```

- Let the best validation accuracy be larger than 80%.

Task 2 (sincnet_pruning.ipynb)

Prune SincNet

- Determine the pruning rate (percentage of total channel will be pruned).

```
pruning_rate =
```

Don't set too large at the first iteration.

- Observe channel amount and accuracy before and after pruning.

```
(features): ModuleList(  
  (0): _Layer(  
    (conv0): Conv1d(38, 38, kernel_size=(25,), stride=(2,), groups=38)  
    (conv1): Conv1d(38, 213, kernel_size=(1,), stride=(1,))  
    (relu): ReLU()  
    (bn): BatchNorm1d(213, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (pool): AvgPool1d(kernel_size=(2,), stride=(2,), padding=(0,))  
  )
```

After pruning

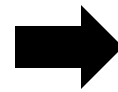
Task 2 (sincnet_pruning.ipynb)

Prune SincNet

- Observe parameter of network and accuracy before and after pruning.

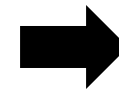
Validation accuracy: 95.44
Number of parameter: 0.27M

Before pruning



?

After pruning



?

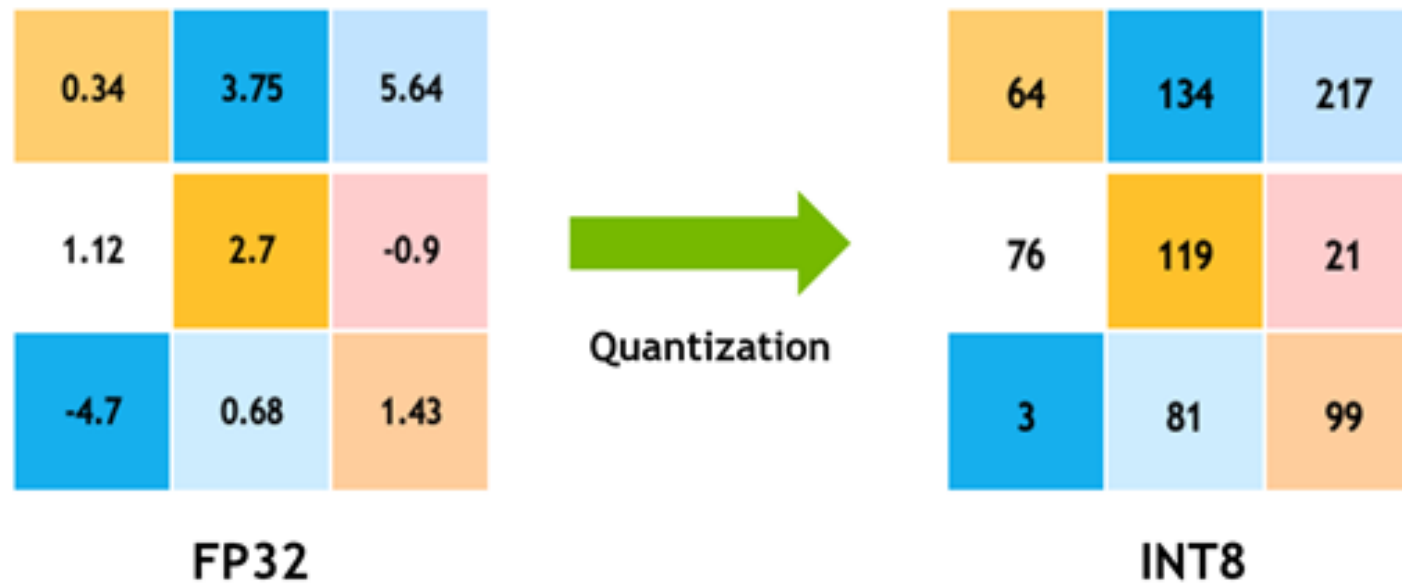
Fine-tuning

- Prune the model that have been fine-tuned repeatedly for twice.

```
# Load model
net = model.SincNet().cuda()
model_path = './Checkpoint/SincNet_best.pth.tar'
# you could load the model after pruning and fine-tune to prune again
# model_path = './Checkpoint/SincNet_finetune.pth.tar'
```

Concept (quantization)

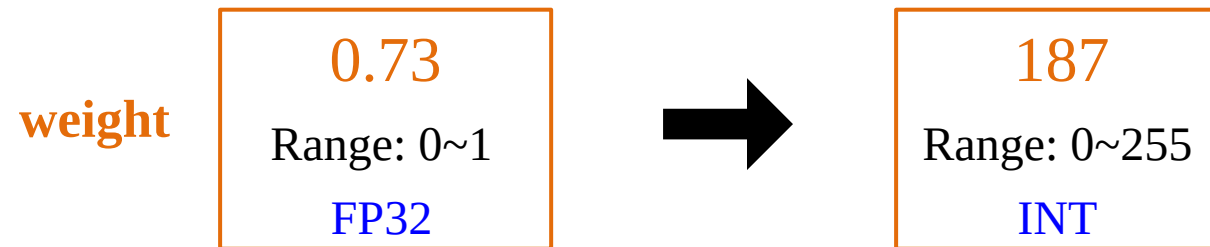
Quantization stores tensors at lower bit-widths than floating point. This allows for a more compact model representation and the use of high performance vectorized operations on many hardware platforms.



Method (quantization)

Change the range and precision of initial weight.

Ex: `number_of_bits = 8`



```
class quantize(torch.autograd.Function):  
  
    @staticmethod  
    def forward(ctx, input_, num_of_bits):  
        n = 2 ** (num_of_bits-1)  
        input_ = input_ * n  
  
        return input_.round() / n
```

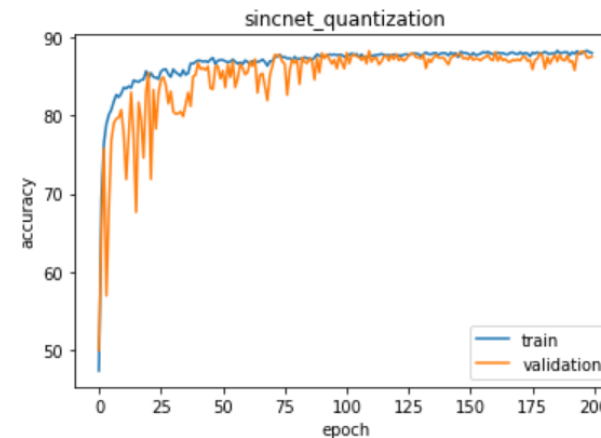
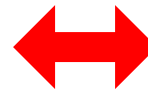
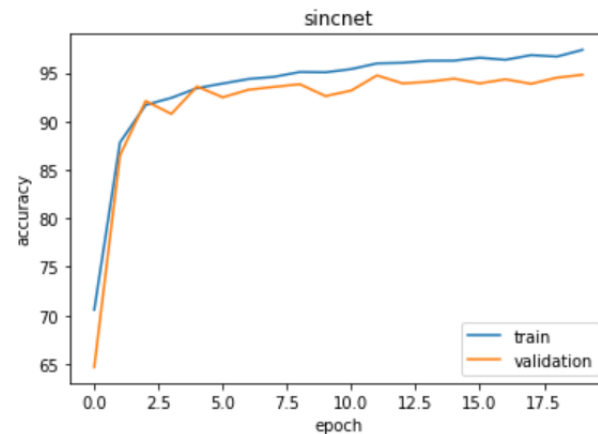
Task 3 (sincnet_quat.ipynb)

Train SincNet_Quat

- Determine super parameter.

```
EPOCH = |  
BATCH_SIZE =  
LR =  
Weight_decay =
```

- Let the best validation accuracy be larger than 70%.
- Observe the training process between with and without quantization.



Task 4 (Report)

- Report should include
 - The screenshot of training result from task 1 (score : 10%)
 - The screenshot of training result from task 2 (score : 10%)
 - Explain **Pruning** briefly (What/Why/How) (score : 20%)
 - The screenshot of training result from task 3 (score : 10%)
 - Explain **Quantization** briefly (What/Why/How) (score : 20%)
 - Compare the difference from above methods of compression (score : 30%)
- **Submit deadline : 1 week only (2024/11/11, 23:59)**
- **You only need to submit your report file**
- **Please strictly follow the naming rule !!!!!**
- **學號_lab4_report.pdf**

Reference

- PRUNING TUTORIAL
 - https://pytorch.org/tutorials/intermediate/pruning_tutorial.html
- Rethinking the Value of Network Pruning
 - <https://github.com/Eric-mingjie/rethinking-network-pruning>
- SincNet
 - <https://github.com/mravanelli/SincNet>
- Post-training Static Quantization — Pytorch
 - <https://medium.com/@sanjanasrinivas73/post-training-static-quantization-pytorch-37dd187ba105>
- Speech commands
 - <https://arxiv.org/abs/1804.03209>

Shut down your kernel !!!!!

- Remember to **SHUTDOWN** the kernel to release resources for others